

Отчёт по лабораторной работе № 4 ▾

Название работы: Коррекция истории: amend, reset, revert, stash

Студент: Дурак Рамазан Угур

Группа: AT-15

Дата выполнения: 06.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Создать репозиторий и сделать три коммита (A, B, C)	☒	<pre>\$ mkdir lab4_history && cd lab4_history \$ git init Initialized empty Git repository in C:/Users/ugur/lab3_remote/lab4_history/.git/ \$ echo "A" > a.txt; git add .; git commit -m "A" warning: in the working copy of 'a.txt', LF will be replaced by CRLF the next time Git touches it [master (root-commit) 5aa163a] A 1 file changed, 1 insertion(+) create mode 100644 a.txt \$ echo "B" > b.txt; git add .; git commit -m "B" warning: in the working copy of 'b.txt', LF will be replaced by CRLF the next time Git touches it [master 7ebc4c5] B 1 file changed, 1 insertion(+)</pre>

Шаг	Описание	Вып.	Команды / вывод
			<pre> create mode 100644 b.txt \$ echo "C" > c.txt; git add .; git commit -m "C" warning: in the working copy of 'b.txt', LF will be replaced by CRLF the next time Git touches it warning: in the working copy of 'c.txt', LF will be replaced by CRLF the next time Git touches it [master cffc046] C 1 file changed, 1 insertion(+) create mode 100644 c.txt </pre>
2	Использовать git commit --amend, чтобы добавить забытый файл D в последний КОММИТ	✓	<pre> \$ git commit --amend -m "C и D (забыл D)" [master 773d17b] C и D (забыл D) Date: Thu May 7 03:24:52 2026 +0500 2 files changed, 2 insertions(+) create mode 100644 c.txt create mode 100644 d.txt </pre>
3	Выполнить git reset --soft HEAD~1, затем снова сделать коммит	✓	<pre> \$ git commit --amend -m "C и D (забыл D)" [master 773d17b] C и D (забыл D) Date: Thu May 7 03:24:52 2026 +0500 2 files changed, 2 insertions(+) create mode 100644 c.txt create mode 100644 d.txt </pre>

Шаг	Описание	Вып.	Команды / вывод
4	Сделать коммит E, затем git reset HEAD~1 (mixed)	✓	git reset --soft HEAD~1 git commit -m "C и D правильно" [master 8146740] C и D правильно 2 files changed, 2 insertions(+) create mode 100644 c.txt create mode 100644 d.txt
5	Выполнить git reset --hard HEAD~1, чтобы потерять коммит E	✓	\$ git reset --hard HEAD~1 HEAD is now at 5aa163a A
6	Сделать ошибочный коммит и отменить его через git revert	✓	echo "Ошибочный коммит" > mistake.txt git add mistake.txt git commit -m "Добавить mistake.txt" [master 99fa73d] Добавить mistake.txt 1 file changed, 1 insertion(+) create mode 100644 mistake.txt git revert HEAD --no-edit [master a6ec222] Revert "Добавить mistake.txt" 1 file changed, 1 deletion(-) delete mode 100644 mistake.txt
7	Спрятать незакоммиченные изменения через git stash, сделать другой коммит, затем применить stash (git stash pop)	✓	echo "Недоделанная работа" > wip.txt git add wip.txt git stash save "WIP перед переключением" Saved working directory and index state On master: WIP перед переключением git status On branch master nothing to commit, working tree clean

Шаг	Описание	Вып.	Команды / вывод
			<pre>echo "Другое изменение" > other.txt; git add .; git commit -m "Other" [master 8757c31] Other 1 file changed, 1 insertion(+) create mode 100644 other.txt git stash pop On branch master Changes to be committed: new file: wip.txt Dropped refs/stash@{0} (4593aef9bee9369b68e4411 0f73e5230e130bd35)</pre>

2. Самостоятельное задание

Условие:

Сделайте 3 коммита, измените сообщение второго через `git rebase -i`, спрячьте файл, сделайте `git reset --hard` на два коммита назад, затем найдите потерянный коммит через `git reflog` и вернитесь к нему.

Выполнение (ход действий и результат):

```
echo "1" > file1.txt && git add file1.txt && git commit -m "Commit 1"
echo "2" > file2.txt && git add file2.txt && git commit -m "Commit 2"
echo "3" > file3.txt && git add file3.txt && git commit -m "Commit 3"
git rebase -i HEAD~3
echo "test" > hidden.txt
git add hidden.txt
git stash
git reset --hard HEAD~2
git reflog
git reset --hard 773d17b
```

3. Ответы на контрольные вопросы

1. Вопрос 1:

Да, можно, но только если эти данные уже были закоммичены. Незакоммиченные изменения будут потеряны навсегда. Для восстановления я использую команду `git reflog`. Она показывает историю всех перемещений указателя HEAD. Я нахожу там

хэш "потерянного" коммита и пишу `git reset --hard <хэш>`, чтобы вернуться в это состояние (как мы и делали в практическом задании).

2. Вопрос 2:

Команду `git revert` нужно использовать, когда изменения уже отправлены в удаленный репозиторий и ветка стала общей для команды. Если сделать `git reset`, то история коммитов перепишется, и у других разработчиков возникнут серьезные конфликты версий. `git revert` действует безопасно: не удаляет старый коммит, а создает новый, который отменяет предыдущие изменения. То есть, локально для себя я использую `reset`, а для публичных веток — только `revert`.

3. Вопрос 3:

Обе команды достают изменения из временного хранилища (`stash`) обратно в рабочую директорию. Главное отличие в том, что `git stash pop` применяет изменения и сразу удаляет их из списка `stash` (я использую это чаще, чтобы список оставался чистым). А `git stash apply` применяет изменения, но оставляет их копию в хранилище (это удобно, если мне нужно применить один и тот же спрятанный код в нескольких разных ветках).

4. Дополнительные замечания (при необходимости)

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.